

COL215: Software Assignment 1

Aditya Narware, Anirudha Saraf

25 August 2024

Name	Entry Number	Group
Aditya Narware	2023CS10805	03
Anirudha Saraf	2023CS10806	03

Table 1: Student Information

Problem Statement

In this assignment, we aim to solve a two-dimensional packing problem by automatically generating a compact physical layout for a gate-level circuit. The layout consists of rectangular logic gates, each with a specified width and height. The goal is to assign locations to all gates in a plane such that no two gates overlap, and the bounding box of the entire circuit, defined as the smallest rectangle enclosing all gates, has the minimum possible area. This problem requires maximizing packing efficiency by minimizing blank space within the bounding box. Notably, the gates cannot be re-oriented or rotated, and connections between gates are ignored for simplification. The result should be an optimized layout that effectively utilizes the available space while adhering to the given constraints.

Algorithm

We are inspired by a collection of algorithms, particularly the skyline algorithm. Our approach involves creating 'bins' and utilizing a bin-packing algorithm. For a given bounding box width, we place gates in decreasing order of their height, followed by their width, and then iterate over the widths to find the one that offers the maximum packing efficiency.

For each placement of gates, our algorithm maintains bins. To place a new bin, we search among all bins from bottom to top and place the gate in the

first bin where it fits, starting from the leftmost position (i.e., the lowest possible x-coordinate and bottommost y-coordinate). We then update the bins accordingly.

When a new gate is placed, two new bins are formed: one with the gate's height and the maximum possible width, and another with the gate's width and the maximum possible height. The height of our bounding box is not pre-limited but is instead determined by the width being iterated over.

Algorithm Steps

1. The input file is read, and the dimensions of the gates are stored.
2. The gates are first sorted in decreasing order of height. This sorted order is maintained for every iteration.
3. We iterate over the widths starting from 1 up to $\sqrt{\text{total area of gates}}$.
4. For a given width, the gates are placed following these steps:
 - (a) The first gate is placed at the origin.
 - (b) Two bins are created:
 - i. The first bin is just to the right of the placed gate, with the same height and the remaining width.
 - ii. The second bin is above the first gate, covering the full width.
 - (c) For the next gate, we check the lowest bin first. If the gate fits in the bin, it is placed leftmost and bottom most within that bin. If it does not fit, the next bin is checked.
 - (d) We continue from the lowest bin upwards.
 - (e) Once a gate is placed in a bin, the bin is further divided into two bins similarly as in the initial step. The upper bin is limited to the previous bin if it was restricted.
 - (f) This process continues until all gates are placed, and the height of the bounding box corresponding to the given width is obtained.
5. While iterating over the widths, the algorithm records the configuration that yields the maximum packing efficiency.
6. The algorithm then repeats the process for heights:
 - (a) The gates are sorted in decreasing order of width followed by height.
 - (b) The bins are created analogously, and the heights are iterated over.
 - (c) Efficiency for this configuration is also recorded.
7. Finally, the packing configuration with the maximum packing efficiency is output.

8. The output file is written using the maximum efficiency configuration and then visualised by the tkinters program.

article graphicx

1 Visualisation

Here is an example of the algorithm in working:

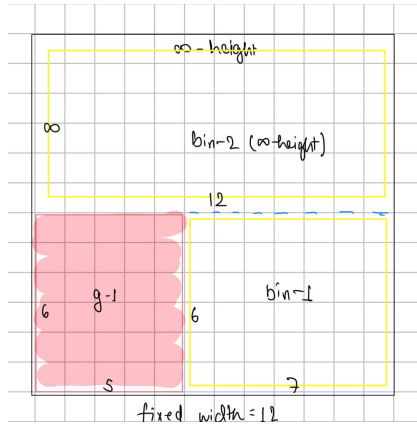


Figure 1: First gate is placed at origin

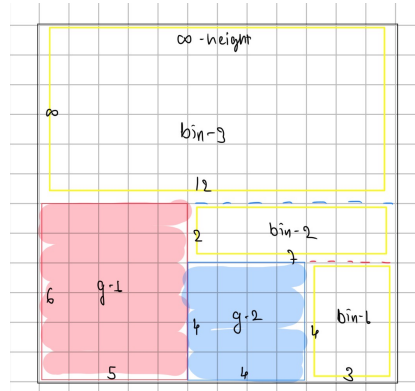


Figure 2: Second gate is first bin (it was sufficiently big)

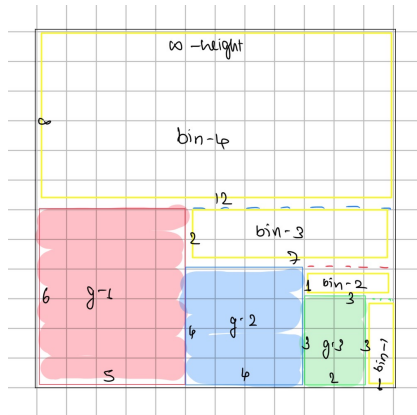


Figure 3: Case: Third gate is placed in bin-1

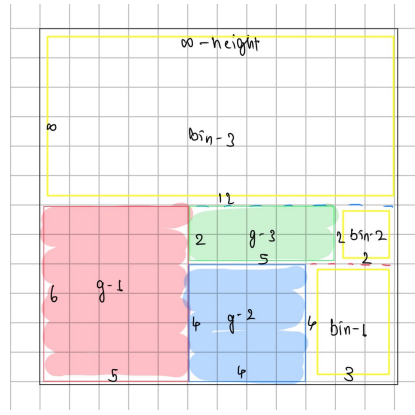


Figure 4: Case: Third gate is placed in bin-2 (bin-1 was not sufficient)

article amsmath

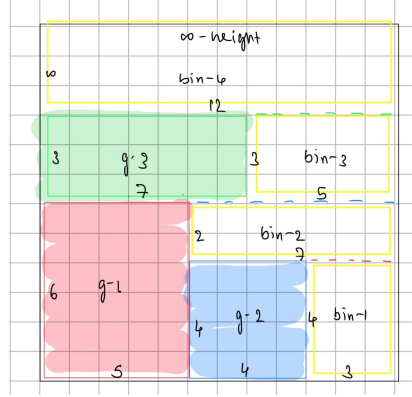


Figure 5: Case: Third gate is placed in bin-3 (neither of bins 1, 2 are sufficient)

2 Time Complexity

In this section, we analyze the time complexity of various functions and algorithms.

2.1 process input function

The function reads the file line by line and processes each line to create a list of blocks. Reading the file and processing each line takes linear time relative to the number of lines (blocks) in the file.

$$O(n)$$

2.2 process output function

This function writes the bounding box dimensions and block positions to a file. Writing each block's information and the bounding box requires linear time relative to the number of blocks.

$$O(n)$$

2.3 sorter function

The sorter function sorts the blocks based on the specified mode (width, height, max side, or area). The sorting operation has a time complexity of $O(n \log n)$ due to the sorting algorithm used.

$$O(n \log n)$$

2.4 pack function

The **pack** function is responsible for packing a set of rectangular blocks into bins, with two main modes: 'vertical' and 'horizontal'. The function involves the following steps:

1. Sorting: The blocks are sorted based on their dimensions. Sorting has a time complexity of $O(n \log n)$, where n is the number of blocks.

2. Packing Process:

- For each block, the function iterates over the list of bins to find a suitable bin where the block can fit. In the worst case, it checks every bin for every block.
- The number of bins in the worst case can be equal to the number of blocks, n . Thus, the function performs up to n checks for each of the n blocks.

Given these nested iterations, the time complexity for this packing process is $O(n^2)$.

3. Overall Complexity: The dominant term in the complexity analysis is the packing process, which has a time complexity of $O(n^2)$. The sorting step, with $O(n \log n)$, is overshadowed by the packing process. Therefore, the overall time complexity of the **pack** function is:

$$O(n^2)$$

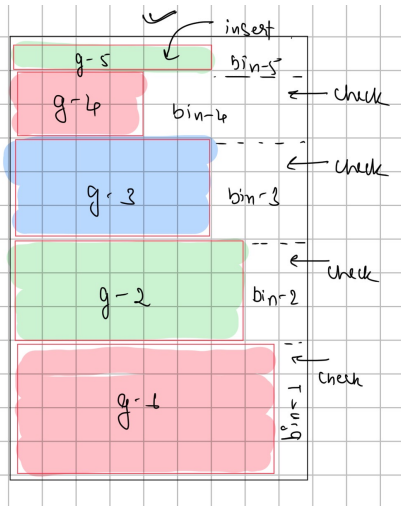


Figure 6: The worst scenario

2.5 Time Complexity of the main Function

The `main` function orchestrates the overall execution of the packing algorithm. It performs several operations, but for time complexity analysis, we focus on the dominant aspects:

1. Iterating Over Widths:

- The `main` function contains a loop that iterates over possible widths for packing. The number of iterations in this loop is proportional to the square root of the total area of the blocks, denoted as $\sqrt{A}$, where A is the total area of the gates.
- For each width, the `pack` function is called to compute the packing arrangement. As established in the previous analysis, the time complexity of the `pack` function is $O(n^2)$, where n is the number of blocks.

2. Overall Complexity:

- Since the `pack` function is called for each width, and there are $\sqrt{A}$ different widths to test, the overall time complexity of the `main` function is the product of the time complexity of the `pack` function and the number of iterations.
- Therefore, the time complexity of the `main` function is $O(n^2 \cdot \sqrt{A})$.

It is noteworthy that the actual runtime is often faster than this theoretical upper bound. This is because the worst-case scenario, where every width would indeed require checking all bins, does not occur in practice. As the width increases, the packing process generally becomes faster due to larger available bins and reduced complexity in fitting subsequent blocks.

$$O(n^2 \cdot \sqrt{A})$$

article amsmath graphicx booktabs float placeins

3 Results

Below are some general results of the algorithm in practice. The tests were conducted with height and width limits set to 100 units. The analysis is based on a randomly generated dataset. Time measurements were recorded using the PyCharm IDE.

Our results show that the algorithm works very well in practice. For randomly generated datasets, it typically achieves packing efficiencies between 90% and 99%. The algorithm usually runs in under 10 seconds, which highlights its speed and effectiveness. As the number of input gates increases, the packing efficiency often improves. This is because there is more room for optimization with more data, and the algorithm benefits from this variety. Overall, the algorithm proves to be both fast and efficient in various situations.

Number of Gates	Time to Run (seconds)	Packing Efficiency
25	0.02	0.91
50	0.04	0.91
100	0.14	0.94
250	0.70	0.96
500	1.91	0.97

Table 2: Performance and efficiency results: Random Data

Number of Gates	Time to Run (seconds)	Packing Efficiency
25	0.02	0.91
50	0.04	0.94
100	0.08	0.95
250	0.20	0.97
500	0.36	0.98

Table 3: Performance and efficiency results: Random Low Variance Data

Number of Gates	Time to Run (seconds)	Packing Efficiency
25	0.02	0.90
50	0.05	0.94
100	0.16	0.96
250	0.75	0.98
500	2.20	0.98

Table 4: Performance and efficiency results: Random High Variance Data

Number of Gates	Time to Run (seconds)	Packing Efficiency
25	0.01	0.75
50	0.02	0.87
100	0.06	0.90
250	0.29	0.94
500	0.88	0.96

Table 5: Performance and efficiency results: Extreme Aspect Ratio Data

Number of Gates	Time to Run (seconds)	Packing Efficiency
25	0.02	0.94
50	0.05	0.94
100	0.16	0.96
250	0.80	0.97
500	2.54	0.98

Table 6: Performance and efficiency results: Random Squares

Case(inputs)	Their output area	Our output area
1(3)	110	110
2(3)	30	30
3(10)	1800	1800
4(5)	80	90
5(35)	500	518

Table 7: Performance in provided sample cases

4 Sample Test Cases provided

In the provided sample test cases we compare the provided output and our output bounding box areas. These are some great satisfactory results almost matching. (Table 7)

5 Test Cases

This section describes the various test cases used to evaluate the algorithm’s performance and efficiency. The aim is to cover a range of scenarios to ensure the robustness of the algorithm.

5.1 Test Case 1: Random Data

This test case involves randomly generated data where the dimensions of the gates range from 1 to 100 units in both width and height. The goal is to evaluate the algorithm’s efficiency and performance with arbitrary data sets.

5.2 Test Case 2: Random Low Variance Data

In this test case, the gates are generated with low variance in dimensions, providing a more uniform dataset. This setup is used to assess how well the algorithm performs with less variation in gate sizes.

5.3 Test Case 3: Random High Variance Data

Here, the gates are generated with high variance in dimensions, creating a dataset with significant differences in gate sizes. This test case helps evaluate the algorithm’s handling of highly varied data.

5.4 Test Case 4: Extreme Aspect Ratio Data

This test case includes gates with extreme aspect ratios, where the width and height of the gates vary greatly. The aim is to test the algorithm’s effectiveness in managing gates with very different proportions.

5.5 Test Case 5: Random Squares

In this test case, the gates are squares with randomly chosen side lengths. This scenario is used to test the algorithm's performance with uniformly shaped gates.

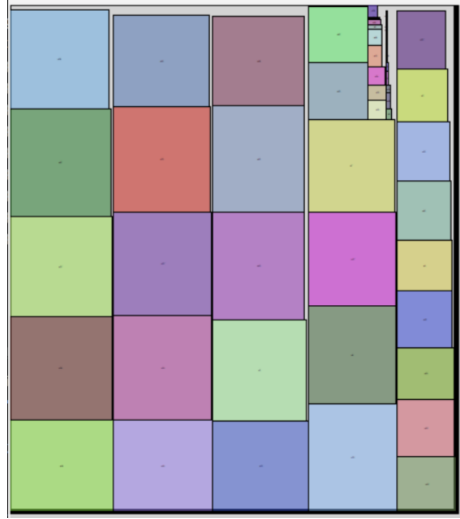


Figure 7: Sample output: High Variance

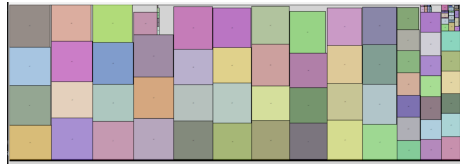


Figure 8: Sample output: Random

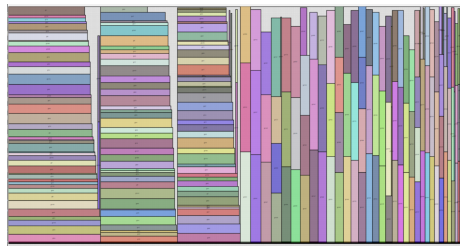


Figure 9: Sample output: High Aspect Ratio

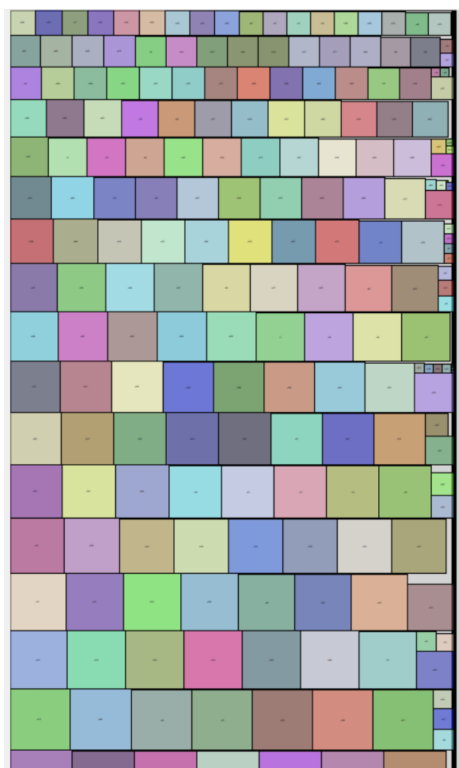


Figure 10: Sample output: Squares



Figure 11: Sample output: Low Vari-
ance